비전 · 딥러닝 — 부록 A 실전

전이학습

남이 사진 128만 장으로 배운 눈을 빌려, 내 사진 몇 장으로 분류기를 만든다

1차시 · 90분
남의 눈을 빌리기

2차시 · 90분
빌린 눈을 살짝 고쳐 쓰기

그다음 · 4시간
해커톤 — Flowers-102

1차시 · 90분

남의 눈을 빌리기

사진을 많이 보고 자란 모델의 “보는 눈”을 빌려 와서, 내 사진 몇 장으로 분류기를 만든다.

1차시 · 오늘의 목표와 흐름

끝나면 이 세 가지를 말할 수 있다

1. 전이학습이 무엇이고 왜 사진이 적을 때 쓰는지

2. 몸통(눈)과 머리(판단)가 각각 하는 일

3. 빌린 눈을 쓸 때 맞춰야 하는 것 (크기 · 정규화)

시간	하는 일
0–15분	도입 · 비유
15–35분	몸통과 머리
35–60분	실험 보기 · 예측
60–80분	따라 하기 실습
80–90분	정리 · 퀴즈

도입 · 질문

처음 보는 꽃 102종,
한 종에 사진 10장씩 보여 주면
여러분은 구별할 수 있을까?

그럼 아무것도 모르는 컴퓨터는?

도입 · 비유

신입을 뽑을까, 경력직을 뽑을까

신입 = 처음부터 학습

보는 법부터 우리 일까지 전부 가르쳐야 한다. 가르칠 자료(사진)가 아주 많이 필요하다.

경력직 = 전이학습

보는 법은 이미 안다(다른 회사 = ImageNet 에서 배움). **우리 일만** 알려 주면 된다. 자료가 적어도 된다.

전이학습: 다른 일에서 배운 것을 **옮겨 와서(전이)** 내 일에 쓰는 것.

도입 · 정말 차이가 날까

같은 모델, 같은 사진 2,000장, 같은 학습 횟수

신입 (처음부터)

44%

경력직 (전이학습)

83%

부록 7차시 실측 · CIFAR-10 10종 · 학습 2,000장 · ResNet18 · 12 epoch · 검증 정확도 0.445 vs 0.826

용어 · 오늘 쓸 다섯 단어

비유로 먼저, 진짜 이름은 그다음

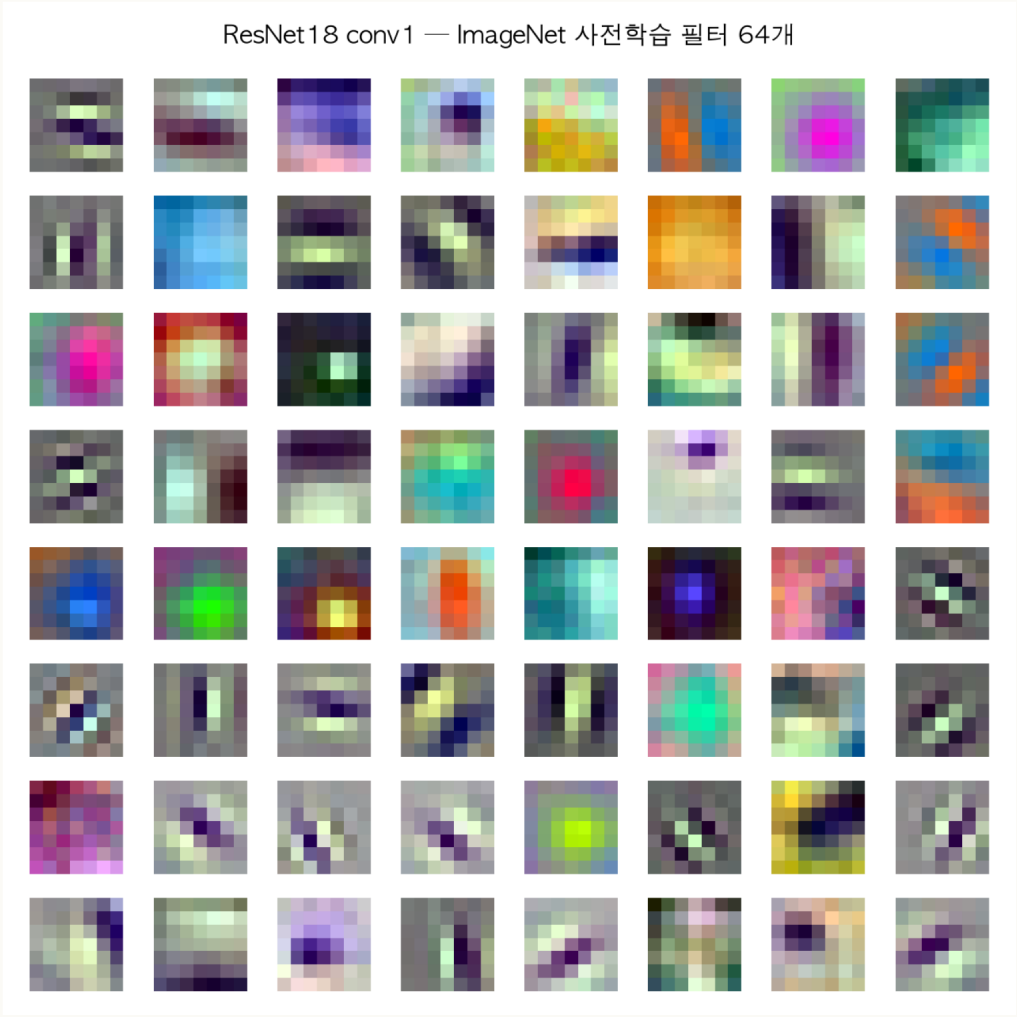
단어	비유	실제로는
사전학습	경력직의 지난 경력	ImageNet 128만 장으로 미리 학습해 둔 가중치
몸통	보는 눈	CNN 의 합성곱 층들 — 사진에서 무늬·모양을 뽑는다
특징	눈이 쓴 메모	몸통이 사진 한 장을 요약한 숫자 512개
머리	판단하는 사람	마지막 층(fc) — 메모를 보고 답(클래스)을 고른다
얼리기	“그 부분은 손대지 마”	학습 중에 그 가중치를 바꾸지 않는 것

눈은 그대로 빌리고, 판단하는 사람만 바꾼다



원래 머리는 ImageNet 의 **1,000종** 중에서 고른다. 우리는 **10종**(또는 꽃 102종)이니 쓸 수 없다.

그래서 머리만 떼고 **우리 클래스 수에 맞는 새 머리**를 단다.



전개 ① · 눈 속을 들여다보기

눈의 첫 단계는 줄무늬와 색을 본다

몸통 맨 앞 층이 찾는 무늬 64개를 그대로 그렸다.

개도, 자동차도 아닌 **어떤 사진에나 있는 것** — 그래서 꽃 사진에도 쓸모가 있다.

뒤쪽 층으로 갈수록 이것들을 조합해 “꽃잎”, “바퀴” 같은 더 큰 모양을 본다.

확인 질문 · 옆 사람과 30초

여기까지 이해했나

Q1. 사전학습 모델을 가져왔을 때 머리는 왜 바꿔야 하나?

Q2. ImageNet 에 꽃 사진이 거의 없어도 꽃 분류에 도움이 되는 이유는?

전개 ② · 실험 · 먼저 예상해 보기

빌린 눈은 정말 쓸모 있을까

A. 훈련된 눈

ImageNet 으로 배운 몸통

B. 무작위 눈

구조는 같고, 가중치는 아무렇게나

둘 다 사진 2,000장을 메모(숫자 512개)로 바꾸고, 그 메모로 **로지스틱 회귀**(머신러닝 시간에 배운 그것)를 학습한다. 신경망은 학습하지 않는다.

A 와 B, 각각 몇 % 맞힐까? 종이에 적어 보자.

전개 ② · 실험 결과

신경망 학습 없이 76% — 눈이 거의 다 했다

A. 훈련된 눈

76%

B. 무작위 눈

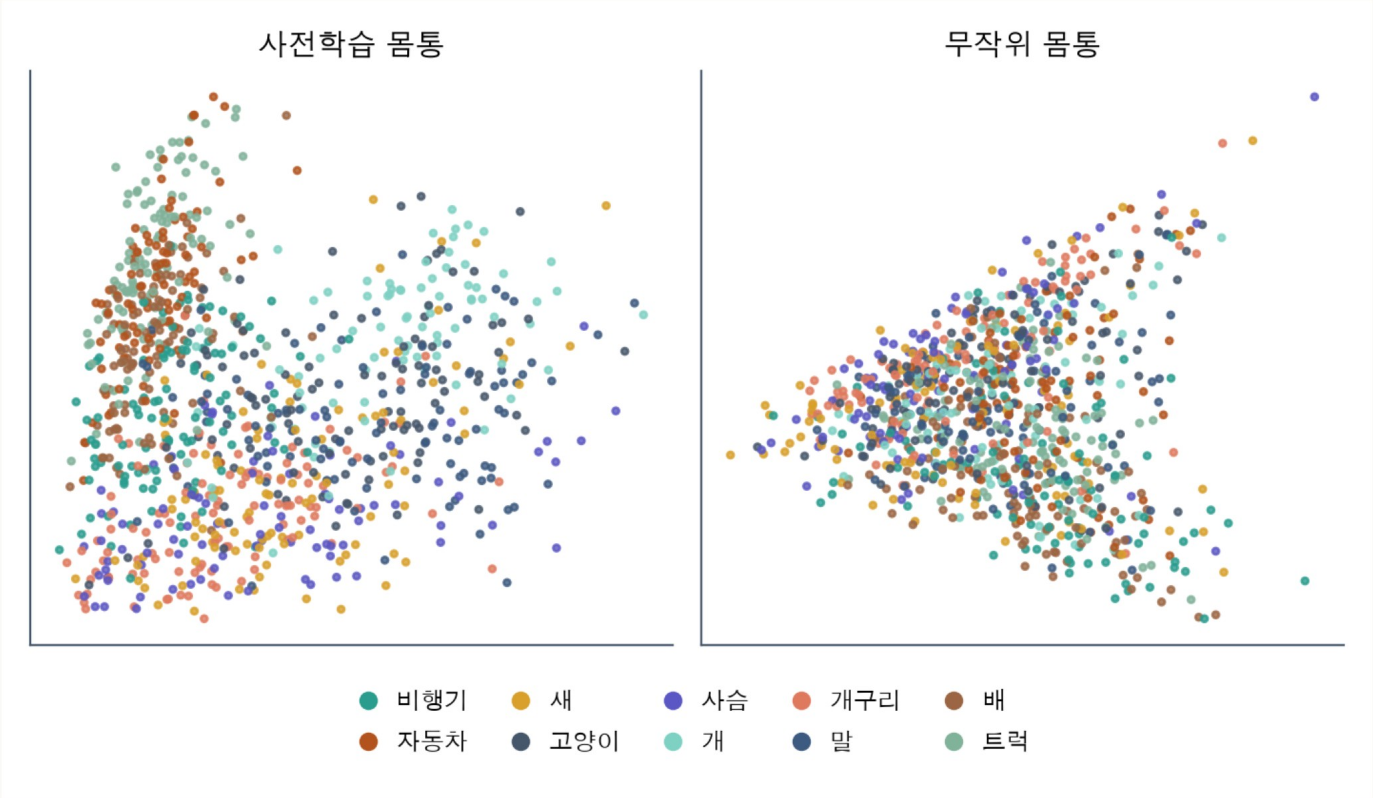
30%

같은 구조, 같은 512개 숫자. 차이는 **눈이 배운 경력** 하나다.

uv run code/09_features.py 의 [A] · CIFAR-10 학습 2,000장 · 검증 2,000장 · 96px · 검증 정확도 0.758 vs 0.300

전개 ② · 메모를 지도에 찍어 보기

훈련된 눈은 비슷한 사진에 비슷한 메모를 쓴다



점 하나 = 사진 1장 · 숫자 512개를 2개로 줄여(PCA) 평면에 찍었다 — figs/09_pca.png

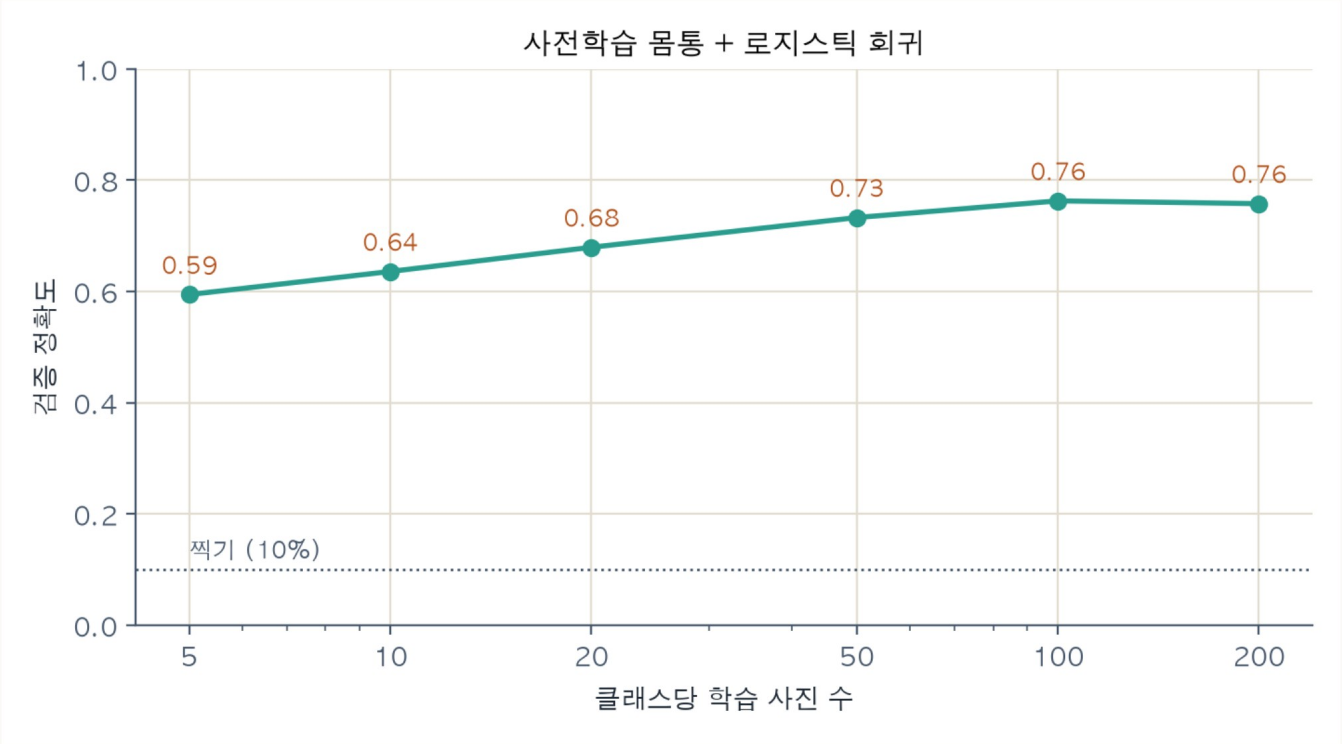
전개 ② · 예상해 보기

한 종에 사진 5장만 주면 훈련된 눈 + 로지스틱은 몇 % 맞힐까?

힌트: 200장일 때 76%, 찍으면 10%

전개 ② · 결과

5장으로 59% — 찍기의 여섯 배



사진이 적을수록 **빌려 온 눈의 값**이 크다. 해커톤 꽃 데이터는 한 종에 **10장** — 이 그래프의 두 번째 점이다.

uv run code/09_features.py 의 [B] · 같은 메모에서 클래스당 n장만 남겨 로지스틱 · figs/09_shots.png

전개 ③ · 빌린 눈에 맞춰 보여 주기

눈이 배울 때 본 모습 그대로 보여 준다

① 크기

ImageNet 사진은 224px. 32px 짜리 작은 사진은 키워서 넣는다.

32px 55%
160px 81%

② 밝기 맞추기 (정규화)

ImageNet 학습 때 쓴 평균·표준편차로 픽셀 값을 맞춘다.

맞춤 76%
안 맞춤 62%

새 정보는 하나도 안 넣었다. **보여 주는 방식**만 바꿨을 뿐이다.

uv run code/09_features.py 의 [C] · [D] · 훈련된 눈 + 로지스틱 · 0.546 / 0.813 · 0.758 / 0.619

코드 · 세 단계

빌리고, 메모를 뽑고, 아는 분류기에 넣는다

① 눈 빌리기

```
model = resnet18(weights="IMAGENET1K_V1")
model.fc = nn.Identity()  # 머리 떼기
model.eval()              # 평가 모드
```

② 메모 뽑기

```
with torch.no_grad():
    memo = model(images)  # (사진 수, 512)
```

③ 분류기

```
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
clf.fit(memo_train, y_train)
```

code/09_features.py — body(), extract(), logistic() 를 줄여 옮김

실습 · 20분 · 따라 하기

한 줄씩 바꾸고, 표를 채운다

0. uv run code/09_features.py 를 그대로 돌려 [A] 표를 확인한다

1. MAIN_SIZE = 96 → **64** 로 바꿔 다시 돌린다

2. 다시 **160** 으로 바꿔 돌린다

3. KNN_K = 5 → **1** 로 바꿔 kNN 줄을 본다

바꾼 것	로지스틱	kNN
그대로	0.758	0.672
64px	____	____
160px	____	____
k = 1	—	____

한 번 돌리는 데 맥 M 시리즈 약 1분 · 시간이 남으면 “심화” 슬라이드의 C 바꾸기

심화 (선택) · 시간이 남을 때

로지스틱도 외운다 — 규제 C 로 막기

C (작을수록 규제가 셈)	학습	검증	간격
0.001	0.848	0.778	0.070
0.01	0.942	0.787	0.155
0.1	0.998	0.776	0.222
1 (기본값)	1.000	0.758	0.242

부록 4차시의 α 와 같은 손잡이다(C = 1/α). 단, 이 표를 보고 C 를 고르면 검증을 엿본 것 — 고를 때는 교차검증으로.

정리 · 10분

오늘의 세 문장

1. 전이학습은 경력직 채용 — 보는 눈(몸통)은 빌리고, 판단(머리)만 새로 단다

2. 빌린 눈의 메모만으로 76% — 한 종 5장으로도 59%

3. 눈이 배울 때 본 모습(크기·정규화)으로 보여 줘야 제 실력이 나온다

나가기 전 퀴즈 — ① 몸통과 머리 중 새로 다는 것은? ② 정규화를 빼면 어떻게 되나? ③ 다음 시간 예고: 눈도 조금 고쳐 쓰면?

2차시 · 90분

빌린 눈을 살짝 고쳐 쓰기

이름은 **미세조정(fine-tuning)**. 어디까지 고칠지, 얼마나 세게 고칠지 두 가지만 정하면 된다.

2차시 · 오늘의 목표와 흐름

끝나면 이 세 가지를 할 수 있다

1. 얼리기·풀기·학습률을 내 말로 설명한다

2. 몸통을 위층부터 풀고, 머리보다 작은 학습률을 준다

3. 사진 수를 보고 “처음 해 볼 설정”을 고른다

시간	하는 일
0–10분	1차시 복습 · 비유
10–30분	손잡이 ① 어디까지
30–45분	손잡이 ② 얼마나 세게
45–55분	사진이 적을 때 · 코드
55–80분	따라 하기 실습
80–90분	정리 · 해커톤 예고

복습 · 1차시 한 장

눈은 그대로 두고, 판단만 새로 했다

몸통 = 눈
얼림 — 전혀 안 바꿈

→

머리 = 판단
새로 학습

76~79%

여기가 천장이었다. 오늘의 질문: **눈도 우리 사진에 맞게 조금 고치면** 더 오를까?

비유 · 경력직 재교육

경력직을 우리 회사 방식으로 재교육할 때

① 어디까지?

기본기(아래층)는 그대로 두고, **세부 업무(위층)**만 다시 가르친다

② 얼마나 세게?

너무 몰아붙이면 원래 실력까지 잊는다. **살살** 고친다

오늘은 이 두 손잡이를 하나씩 돌려 보며 숫자로 확인한다.

용어 · 오늘 쓸 네 단어

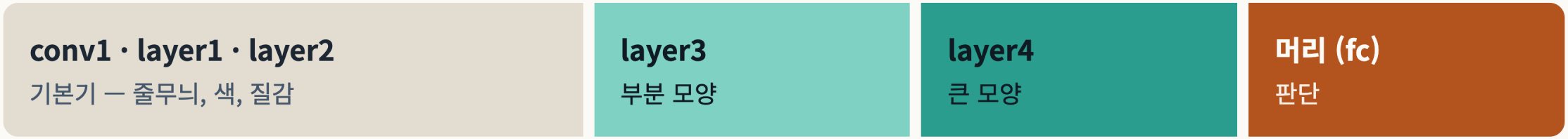
비유로 먼저, 코드 이름은 그다음

단어	비유	코드에서는
얼리기	“이 부분은 재교육 안 함”	p.requires_grad = False
풀기	“이 부분도 다시 가르침”	p.requires_grad = True
학습률	한 번에 고치는 폭	옵티마이저의 lr (예: 1e-3 = 0.001)
미세조정	경력직 재교육	사전학습 가중치에서 출발해 몸통도 학습

1e-3 = 0.001, 1e-4 = 0.0001 — 숫자가 작을수록 살살 고친다.

손잡이 ① · 어디까지 풀까

ResNet18 의 몸통은 층 다섯 덩어리



설정	다시 가르치는 곳
머리만	머리 (1차시와 같은 상태)
layer4 부터	layer4 + 머리
layer3 부터	layer3 + layer4 + 머리
전부	몸통 전체 + 머리

예상해 보기

많이 풀수록 좋을까?
“전부”가 1등일까?

머리만 / layer4 부터 / layer3 부터 / 전부 — 순위를 적어 보자

손잡이 ① · 결과

layer4 하나만 풀어도 거의 다 얻는다

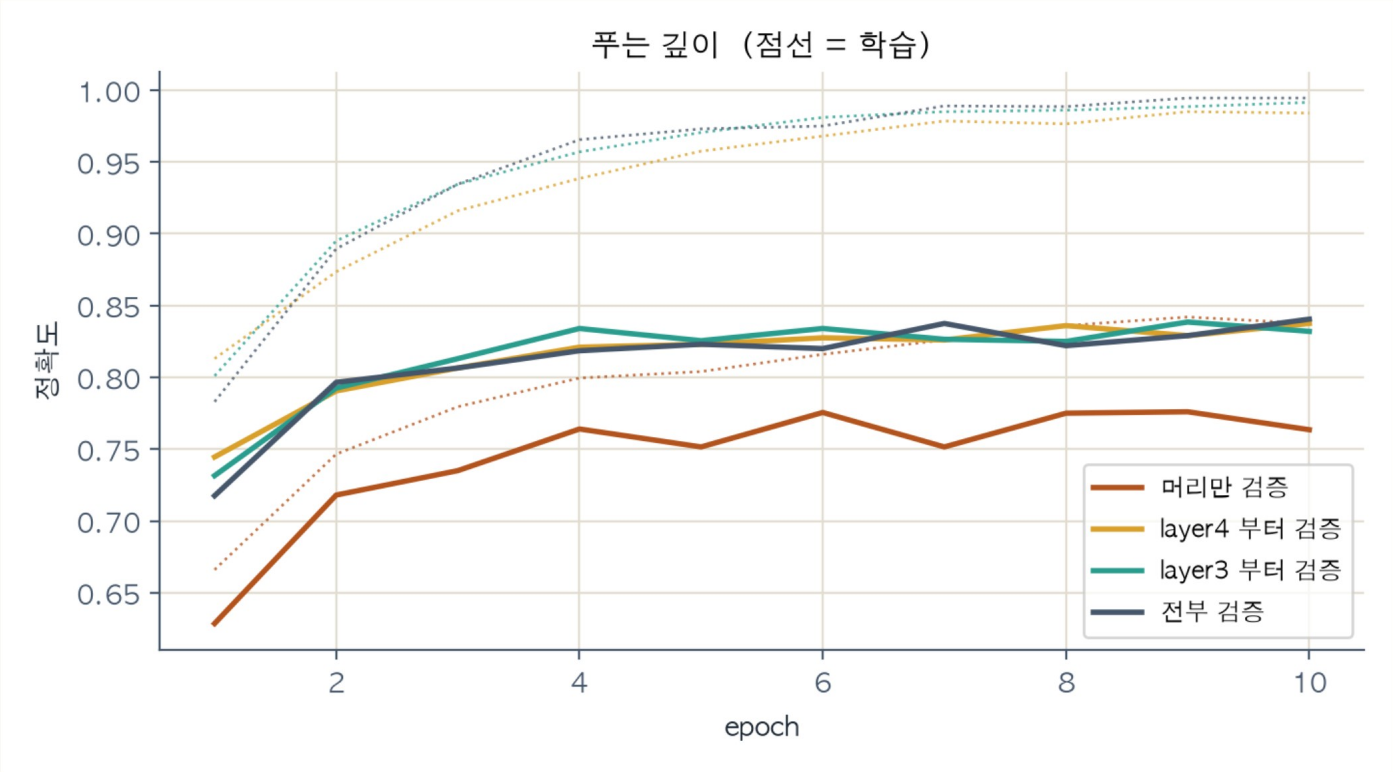


눈을 조금 고치니 **+7%p**. 그 아래까지 더 풀어도 차이는 1%p 안 — **운으로도 생기는 차이**라 순위를 매기지 않는다.

uv run code/10_finetune.py 의 [A] · CIFAR-10 2,000장 · 10 epoch · 머리 1e-3 · 몸통 1e-4 · 막대 길이 = 정확도

손잡이 ① · 학습 과정을 그래프로

고친 셋은 2번 만에 머리만을 앞지른다



가로축 = 학습 반복(epoch) · 실선 = 검증 · 점선 = 학습 — figs/10_depth.png

손잡이 ② · 얼마나 세게 고칠까

몸통 학습률: 머리의 10분의 1 이 가장 좋았다



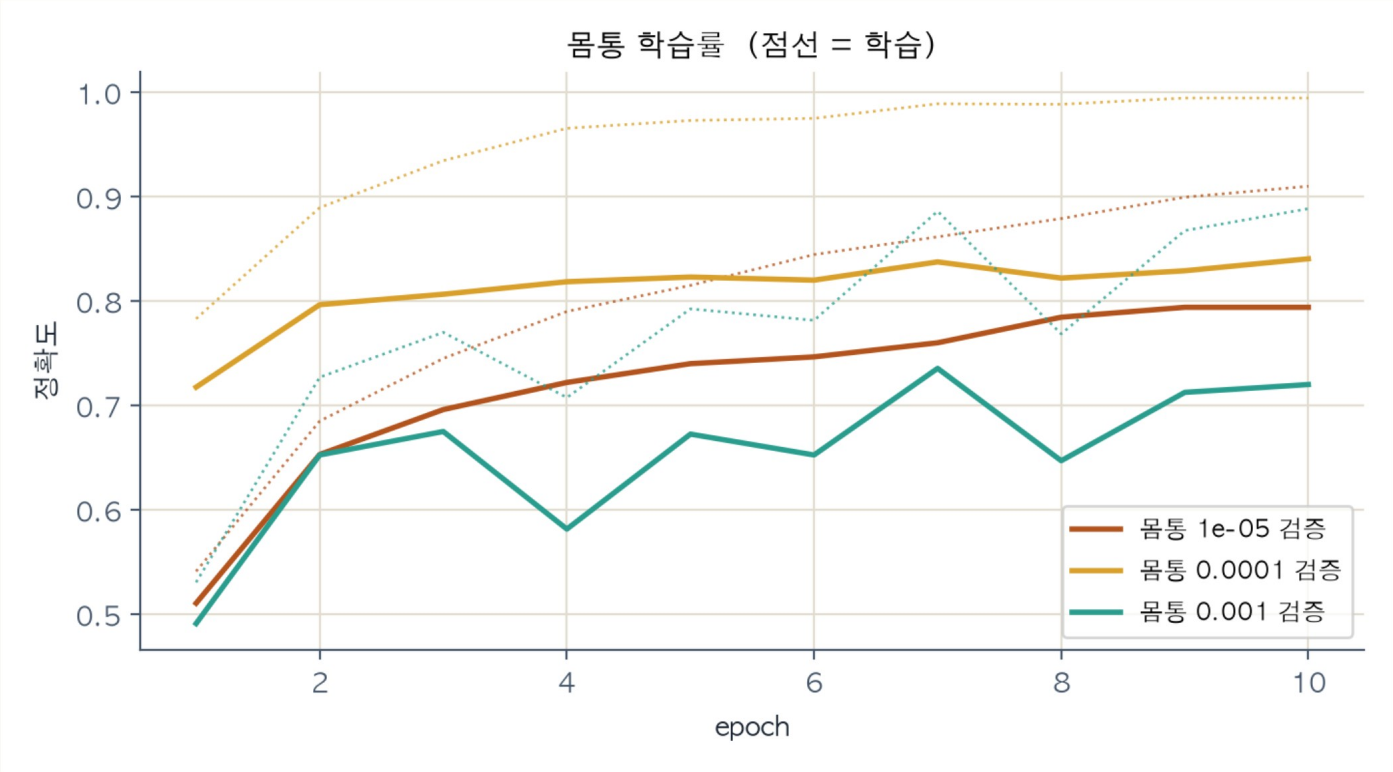
너무 살살: 눈이 거의 안 바뀌어 머리만에 가깝다

너무 세게: 원래 실력을 잊는다 — 머리만(76%)보다도 낮다

uv run code/10_finetune.py 의 [B] · 전부 풂 · 머리는 셋 다 1e-3 · 10 epoch · 막대 길이 = 정확도

손잡이 ② · 그래프로

너무 세게 고치면 학습 점수까지 흔들린다



실선 = 검증 · 점선 = 학습 · 청록 = 1e-3 · 노랑 = 1e-4 · 주황 = 1e-5 — figs/10_body_lr.png

사진이 아주 적으면

한 종에 20장이면 고쳐도 이득이 작다

한 종 200장

머리만 76.4% → 전부 84.1%

+7.7%p

한 종 20장

머리만 61.9% → 전부 63.8%

+1.9%p

20장에서 전부 풀면 학습 사진은 97.5% 맞힌다 — **고친 눈이 200장을 외우기 시작**했다. 사진이 적을수록 증강·짧은 학습 같은 **외우기 막기**가 중요해진다.

uv run code/10_finetune.py 의 [C] · 머리 1e-3 · 몸통 1e-4 · 10 epoch · 증강 켬

코드 · 세 단계

머리 갈기 → 얼리고 풀기 → 학습률 두 개

① 머리 갈기

```
model.fc = nn.Linear(512, 10)  # 우리 클래스 수
```

② 얼리고 풀기

```
for name, p in model.named_parameters():
    p.requires_grad = name.startswith(("layer4", "fc"))
```

③ 학습률 두 개

```
opt = Adam([{"params": 머리, "lr": 1e-3},
            {"params": 푼 몸통, "lr": 1e-4}])
```

순서 주의: ① 머리를 간 **다음에** ③ 옵티마이저를 만든다.

code/10_finetune.py — build(), groups() 를 줄여 옮김 · 실제 코드는 얼린 층의 배치정규화도 고정한다(keep_frozen_bn)

처음 해 볼 순서

막막하면 이 순서로

1단계

머리만 학습해 기준 점수를 잰다

→

2단계

layer4 를 풀고 몸통 1e-4 로

→

3단계

검증이 오르면 더 풀어 보고, 학습만 오르면 **외우기 막기**

매 단계 **하나만** 바꾸고 검증 점수를 적는다. 1%p 안쪽 차이는 시드를 바꿔 다시 잰 뒤에 판단한다.

이상하면 먼저 볼 곳

증상으로 원인 찾기

이런 증상이면	먼저 의심할 것
점수가 찍기 수준	머리를 갈기 전에 옵티마이저를 만들었다
학습 점수까지 들쭉날쭉	몸통 학습률이 너무 크다 (1e-3 → 72%)
학습 ≈100%, 검증은 낮음	사진에 비해 너무 많이 풀었다 → 덜 풀거나 외우기 막기
정확도가 생각보다 한참 낮음	크기·정규화가 사전학습 때와 다르다 (1차시)

실습 · 25분 · 따라 하기

한 줄씩 바꾸고, 표를 채운다

1. EPOCHS = 10 → **5** · [A] 표에서 읽기

2. EPOCHS 를 10 으로 되돌리고 SMALL_PER_CLASS = 20 → **50** · [C] 표에서 읽기

3. BODY_LRS 에 **3e-4** 를 더하기 · [B] 표에서 읽기

바꾼 것	머리만	전부
그대로	0.764	0.841
1. 5 epoch	____	____
2. 한 종 50장	____	____
3. 몸통 3e-4	—	____

다 채우면 질문 하나: **사진이 적을수록, “전부 풀기”의 이득은 어떻게 변하나?**

한 번 돌리는 데 맥 M 시리즈 약 2분 · CPU 만 있으면 짝과 함께 · 끝나면 바꾼 값을 되돌린다

정리 · 10분

오늘의 세 문장

1. 미세조정은 경력직 재교육 — 눈의 위층(layer4)만 고쳐도 +7%p

2. 몸통은 살살 — 머리의 10분의 1. 세게 고치면 원래 실력을 잊는다

3. 사진이 적을수록 고치는 이득은 줄고, 외우기 막기가 중요해진다

나가기 전 퀴즈 — ① requires_grad = False 는 무슨 뜻? ② 몸통 학습률은 머리보다 크게? 작게? ③ 꽃 102종, 한 종 10장이면 처음에 무엇부터?

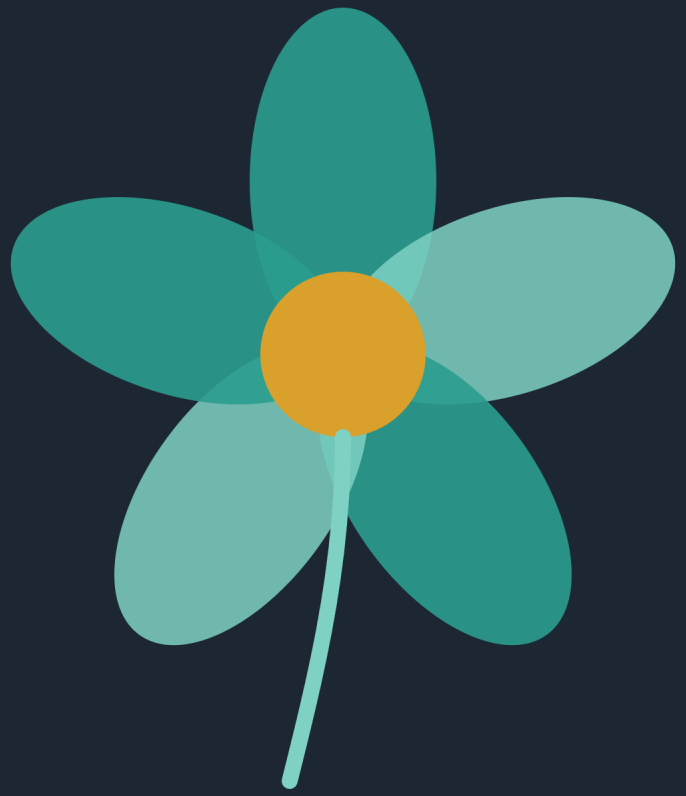
비전 · 딥러닝 — 부록 A 실전

전이학습 해커톤

꽃 102종, 한 종에 사진 10장. 배운 과적합 막기로 시험 정확도를 끝까지 끌어올린다

Flowers-102 · ResNet18

4시간 · [팀 구성]



과제

사진 6,149장의 꽃 이름을 맞힌다

102

꽃의 종류

10

한 종당 학습 사진 수 — **외우기 딱 좋은 양**

6,149

라벨 없는 시험 사진 — 점수는 여기서

모두 같은 ResNet18(ImageNet 사전학습)에서 출발한다. 차이는 **무엇을 학습시키고, 어떻게 막고, 어떻게 고르느냐**에서 난다.

Oxford Flowers-102 (Nilsback & Zisserman, 2008) · 공식 train / val / test 분할 그대로

받는 것

zip 하나 — 사진 세 묶음과 코드 세 파일

폴더	사진	라벨
data/train	1,020	폴더 이름 000~101
data/val	1,020	폴더 이름 000~101
data/test	6,149	없음 — 이것을 맞힌다

학습·검증은 한 종에 10장씩 고르게. 시험은 종마다 **20~238장으로 들쭉날쭉**하다.

```
flowers-hackathon/
  README.md
  baseline.py   # 고친다
  common.py     # 고치지 않는다
  data/
    train/ val/ test/
```

zip 약 340MB · 수업 전에 받아 풀어 둔다 · ResNet18 가중치(45MB)는 처음 실행 때 내려받는다

진행 순서

4시간 — 그중 2시간 40분이 해킹

시간	할 일
0:00 – 0:20	규칙 설명 · 모두 baseline.py 한 번 돌려 보기 (안 도는 노트북은 여기서 잡는다)
0:20 – 3:00	해킹 · 공개 보드 제출은 팀당 10번까지
3:00	마감 · 최종 제출 1개를 고른다 → 비공개 보드 공개
3:10 – 4:00	상위 팀 5분 발표 · 공개 순위와 비공개 순위가 왜 달라졌나

시간은 진행 상황에 맞춰 주최 측이 조정할 수 있다

규칙

같은 출발선 — 모델이 아니라 기법으로 겨룬다

해도 된다

- baseline.py 의 어디든 고치기
- 증강 · 규제 · 스케줄 · 얼리기/풀기
- 검증 사진을 학습에 합치기
- 테스트 시 증강(TTA) · 모델 2개까지 앙상블

하면 안 된다

- resnet18 IMAGENET1K_V1 말고 다른 모델·가중치
- 외부 데이터 · torchvision Flowers102 내려받기
- 입력 224px 초과
- 시험 사진으로 학습 · 손으로 라벨 붙이기

상위 팀은 주최 측 컴퓨터에서 코드를 다시 돌린다. **같은 점수가 나와야 순위가 인정된다.**

README.md 의 규칙이 이 슬라이드와 같다 · 애매하면 해킹 전에 묻는다

시작하기

명령 하나로 학습부터 제출 파일까지

```
$ cd flowers-hackathon
$ uv run baseline.py
장치: mps
epoch  1  검증 0.2755
…
epoch 10  검증 0.7794
최종 검증 정확도 0.7794
제출 파일: …/submission.csv
```

uv 가 torch·torchvision 을 알아서 설치한다. 맥 M 시리즈 약 1분 20초 · CPU 로는 고성능 노트북에서도 5분 넘게

고칠 곳

baseline.py 의 [바꿀 곳] 다섯 군데

바꿀 곳	기준선은 지금
[1] 변환	160px 로 줄여 가운데만 자른다 · 증강 없음
[2] 모델	몸통 전부 얼림 · 새 머리(fc)만 학습
[3] 손실	그냥 cross_entropy
[4] 옵티마이저	Adam 1e-3 · 스케줄 없음 · 가중치 감쇠 없음
[5] 학습 루프	10 epoch · 마지막 epoch 의 모델을 그대로 제출
predict()	사진 한 장에 한 번 예측 — 테스트 시 증강을 넣을 자리

common.py (데이터 읽기 · 정확도 · 제출 파일 검사) 는 고치지 않는다 — 채점도 같은 파일로 한다

출발점

기준선은 검증 77.9% — 시험에서는 더 낮다

검증 1,020장

0.779

공개 보드 3,074장

0.739

같은 모델인데 4%p 차이. 검증은 종마다 10장으로 고르고, 시험은 많이 찍힌 종과 적게 찍힌 종이 섞여 있다 — **검증은 비교하는 자이지, 시험 점수의 예고가 아니다.**

uv run baseline.py · 시드 0 · 맥 M 시리즈 MPS 1분 20초

목표

주최 측 참고 해법은 시험 약 92%

0.74 → 0.92

부록 A 에 나온 기법만 썼고, 맥 M 시리즈에서 학습 2분이 걸렸다. 같은 ResNet18 로 18%p — 무엇을 바꿨을지는 여러분이 찾는다.

참고 해법은 대회가 끝난 뒤 공개한다 · 시드 1개 기준

무기 — 부록 A 에서 가져온다

배운 기법마다 넣을 자리가 있다

기법	넣을 곳	다시 볼 곳
데이터 증강 · 해상도 올리기	[1] 변환	7차시 ② 증강
미세조정 · 층마다 다른 학습률	[2] · [4]	7차시 ③ 전이학습
라벨 스무딩 · Mixup	[3] 손실	7차시 ②
AdamW 가중치 감쇠	[4]	6차시 ② 가중치 감쇠
학습률 스케줄 · 워밍업	[4]	6차시 안정화 기법
조기 종료 · 가장 좋았던 모델 남기기	[5] 학습 루프	6차시 ① 조기 종료
테스트 시 증강 (뒤집어서 한 번 더)	predict()	—

부록 A 슬라이드 PDF: deeplearning/appendix-overfit/과적합막기.pdf · 7차시 코드 code/07_images.py 에서 그대로 옮겨 와도 된다

함정

점수를 깎는 세 가지 실수

평가에 증강

무작위 자르기·뒤집기를 eval_transform 에도 넣으면 검증 점수가 돌릴 때마다 흔들린다. 증강은 **train_transform 에만**

몸통에 큰 학습률

몸통을 풀고 머리와 같은 1e-3 을 주면 검증 0.848 → **0.828** (시드 1개 실측). 몸통은 머리보다 작게 — 얼마나 작게는 재 보고 정한다

마지막 epoch 제출

기준선은 마지막 모델을 낸다. 검증이 가장 좋았던 순간의 가중치를 **저장해 두었다가** 그것으로 예측한다

세 번째는 고칠수록 검증 세트를 엿보는 일이기도 하다 — 그래서 최종 순위는 아무도 보지 못한 비공개 보드로 정한다.

채점

시험 사진은 둘로 나뉘어 있다

공개 보드 · 3,074장

제출할 때마다 점수를 보여 준다. 팀당 **10번**까지

비공개 보드 · 3,075장

끝날 때까지 아무도 모른다. **최종 순위는 이것으로**

어느 사진이 어느 쪽인지는 알려 주지 않는다. 공개 점수가 오른 제출만 골라 쌓으면 — 공개 보드에 **과적합**한 것이다.

마감 때 팀마다 최종 제출 1개를 고른다 · 고르지 않으면 공개 점수가 가장 높았던 것

흩어짐

1%p 안쪽 차이는 같은 점수다

±0.5%p

3,000장 보드에서 정확도 90% 의 표준오차. 운만으로 이만큼 오르내린다

시드 2개+

"이 기법이 먹혔다"고 말하려면 시드를 바꿔 두 번 이상 돌리고 평균 ± 표준편차로 본다

검증 1,020장은 더 작다 — 1장이 **0.1%p**. 검증에서 0.3%p 오른 것은 사진 세 장이다.

√(0.9 × 0.1 / 3,000) ≈ 0.0055 · 2차시 홀드아웃의 운 · 8차시 짝지은 차이

제출

submission.csv 를 팀 이름으로 바꿔 올린다

```
id,label
0a1f93c2,50
0a2b77e1,77
…
# 6,149줄 + 머리줄
```

1. 파일 이름: **팀이름_번호.csv** (예: 해바라기_03.csv)
2. 올릴 곳: [제출 채널]
3. 같이 낼 것: 그 제출을 만든 **baseline.py**

write_submission 이 장수·라벨 범위를 검사한다. 오류 없이 저장됐으면 형식은 맞다.

라벨은 0~101 숫자 (폴더 이름과 같다) · 공개 보드 점수는 주최 측이 grade.py 로 매겨 알려 준다

발표 (상위 팀 5분)

표 하나 — 무엇을 더했고 얼마나 올랐나

단계 (앞 단계 위에 더함)	검증 평균 ± 표준편차	시드 수
기준선	0.779 ± [__]	[__]
+ [기법 1]	[__] ± [__]	[__]
+ [기법 2]	[__] ± [__]	[__]
+ [기법 3]	[__] ± [__]	[__]

말할 것: 가장 크게 오른 단계와 그 이유 · 해 봤는데 안 오른 것

말하지 말 것: 흩어짐보다 작은 차이를 "개선"이라 부르기

이 표는 부록 7차시 작은 CNN 표와 같은 모양이다 — 앞 단계 위에 하나씩 얹는다

자주 묻는 것

시간이 모자랄 때, 막혔을 때

질문	답
GPU 가 없어요	CPU 로 몸통까지 미세조정하면 224px 1 epoch 에 30초 넘게 걸린다(고성능 노트북 실측). 팀 안의 GPU·맥 M 노트북으로 돌린다
DataLoader 오류가 나요	common.py 의 NUM_WORKERS 를 0 으로 (이 값만은 바꿔도 된다)
검증 점수와 공개 점수가 달라요	정상이다. 검증은 종마다 10장, 시험은 들쭉날쭉 — 비교용으로만 쓴다
점수가 1% 근처예요	2차시 함정 첫 줄 — 머리를 바꾸기 전에 옵티마이저를 만들지 않았나

102종이라 찍기는 약 1% · 측정: 맥 M4 Max 의 CPU 16코어 — 보통 노트북 CPU 는 더 느리다

시작

uv run baseline.py

첫 20분 안에 submission.csv 를 한 번 제출해 공개 보드에 이름을 올린다. 그다음부터는 한 번에 하나씩 바꾸고, 검증으로 재고, 적어 둔다.

마감 3:00

질문은 [질문 채널]